eBPFlex: eBPF-based Lightweight Cross-Compartment Policy Enforcement

Abstract

Software compartmentalization effectively confines unknown bugs in applications by logically dividing them into distinct compartments. Several isolation frameworks can already perform automatic compartmentalization for monolithic software. However, vulnerabilities at compartment interfaces can still be exploited to compromise the security of compartmentalized software, and there is a lack of automated tools to mitigate such interface vulnerabilities. In this work, we present eBPFlex, an automated tool that hardens compartment interfaces through a two-phase approach: first inferring security policies using static and dynamic analyses, then generating and deploying eBPF programs to enforce these policies at runtime. Our evaluation on real-world applications demonstrates that eBPFlex can prevent various types of interface vulnerabilities with reasonable performance overhead (less than 10% for most evaluated benchmarks) and protect more than 75% of interface data on average. Overall, our results indicate that eBPFlex can further strengthen the security of compartmentalized applications with minimal developer effort and acceptable runtime overhead, making it a practical solution for compartment interface protection.

ACM Reference Format:

. 2025. eBPFlex: eBPF-based Lightweight Cross-Compartment Policy Enforcement. In . ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Software compartmentalization, also known as software isolation, is an effective design pattern to confine unknown bugs in applications. In this design, an application is logically divided into multiple *compartments*, each assigned its own privileges and interacting with others under the governance of a set of *security policies*. The key advantage of this approach is preventing an unknown bug in a single compartment from compromising the security of the entire application. This contrasts with monolithic system designs, where a single vulnerability can lead to catastrophic results such as a complete system crash or compromise, as exemplified by incidents ranging from the Heartbleed vulnerability [8] in 2014 to the recent massive CrowdStrike outage in 2024 [18].

Despite the clear security benefits enabled by compartmentalization, compartmentalized applications are not bulletproof. In the past few years, multiple research efforts have focused on the attack surface within already compartmentalized applications [4, 7, 24, 32]. In a nutshell, the attack surface in compartmentalized applications mainly exists at compartment interfaces, which are invoked to provide cross-compartment functionality and to exchange data.

Interface safety [25] can be compromised if an interface is not hardened against attacks targeting the interface. Possible attacks are generally categorized into data and control plane [32, 41] in previous work. In a recent study, Lefeuvre et al. used the term Compartment Interface Vulnerabilities (CIVs) to refer to vulnerabilities that can be exploited to abuse compartment interfaces [24]. While the awareness of CIVs has increased in recent compartmentalization research, designing proper mitigation mechanisms remains extremely challenging.

First, there is no clear understanding on what security properties can be compromised by CIVs, and what types of security policies are required to mitigate different CIV types. For instance, untrusted compartments can supply corrupted data to trusted ones, compromising both integrity and confidentiality [19, 24]. To prevent corrupted data from being used by trusted compartments, all data passed from untrusted compartments must be carefully validated with respect to its expected semantics before use [32]. Formalizing this requires a type of security policy that specifies data validation [20, 24].

Second, manually specifying security policies for complex compartmentalization interfaces can be labor intensive and untenable in practice [21]. Thus, automatic security policy inference is needed to provide good mitigation coverage for CIVs [24, 32]. For instance, RLBox [32] extended the C++ type system and automated the isolation procedure with compiler support, where the compiler produces meaningful errors to guide developers through the migration process and automatically handles many security checks. However, to verify that interfaces are properly used and protected against CIVs, it can take up to 3 person-days to isolate a single library and implement the necessary validation functions properly.

Third, enforcing security policies may require extra developer efforts to properly set up and integrate the monitor[31]. Typically, security policies can be enforced by deploying a runtime monitor through a third-party library[41]. Such approaches come with multiple disadvantages from a development perspective. First of all, the integration process is often complex, requiring build-time or deployment-time modifications that are far from trivial. Developers may need to statically or dynamically link the monitoring library or compile the program with instrumentation hooks. This method is inherently intrusive, demanding changes to the program's build configuration or source code, and requires that all modules of the application be compiled against the instrumented library to ensure consistent monitoring. Beyond this significant development overhead, security concerns also arise, as the monitoring library itself must be fully trusted and bug free; any flaw in the monitor can introduce new vulnerabilities or expand the attack surface of the application.

Our work aims to address the above challenges by answering the following questions: (1) What security policies can mitigate CIVs? (2) Can these security policies be inferred automatically? (3) Can

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

we provide a security monitor that can be integrated into existing compartmentalization frameworks, without requiring excessive developer efforts?

To answer Q1, we conduct a comprehensive study of CIVs in existing literature and derive security policies needed to prevent CIV exploitation. For Q2, we analyze each policy and employ static and dynamic analyses to infer many types of security policies that prevent CIVs. For Q3, we build a security monitor using eBPF to enforce these policies. Overall, we design and implement *eBPFlex* (eBPF-based Lightweight Compartment Policy Enforcement), a tool to harden compartmentalized application interfaces against CIVs. Our key contributions are as follows: **(1) We investigate existing CIVs and their impact on interface safety, and design security policies required to address them; (2) We utilize a combination of novel static and dynamic analysis to infer security policies automatically with good protection coverage on the interface data and (3) We automate the enforcement of the inferred policies by generating eBPF programs that serve as security monitors**. And our evaluation shows that with eBPFlex, we are able to protect the majority of interface data (more than 75% across tested benchmarks) with a reasonable overhead of 5 - 10% in most of cases. With these contributions, we believe eBPFlex can largely reduce the manual efforts to provide interface safety in compartmentalized applications, strengthening the security benefits of compartmentalization.

2 Background

2.1 Software Compartmentalization

Software compartmentalization is a fundamental design principle that partitions a program into distinct compartments, also referred to as *protection domains* or *partitions* in prior literature [25]. The formalization of compartmentalization encompasses three key elements: subjects (code), objects (memory regions, sockets, files), and permissions (allowed actions on objects). A compartment is defined as a set of subjects that share identical permission sets.

Principle of least privilege The principle of least privilege [40] mandates that a compartment should be granted only the minimal set of permissions necessary to perform its required task. When correctly applied, this principle ensures that a faulty or malicious compartment can cause only limited damage to the overall system, even if it abuses its granted privileges. When this principle is not obeyed, a compartment may be able to abuse its excessive privileges to attack the rest of the system.

Compartmentalization trust models Trust models define which compartments are trusted and require protection [24–26, 41]. These models fall into three categories: (1) *sandbox*, where untrusted compartments should have reduced privileges to protect the system; (2) *safebox*, where trusted compartments are protected by reducing the rest of the system's privileges; and (3) *mutual distrust*, where least privilege applies to all compartments.

Compartmentalization security properties Compartmentalization can provide different security properties. The fundamental property is *Integrity*, which specifies that a subject cannot write outside of its compartment. This property is enforced by all compartmentalization frameworks. *Confidentiality* specifies that a subject cannot read memory outside of its compartment. However, some compartmentalization works don't enforce confidentiality for performance consideration [42]. Another property is *Availability*, which ensures that a compartment cannot prevent others from executing normally. In general, confidentiality and availability cannot be enforced without enforcing integrity [25, 43].

Compartment interfaces and shared data Compartments communicate with each other using *compartment interfaces*, which are sets of functions invoked to access functionality in other compartments. Shared data refer to the data that are accessed by multiple compartments and whose state is required by the compartments to perform correct execution [16, 21]. By this definition, the data exchanged through the compartments interfaces such as the parameters and return values of interface functions, as well as global data (e.g., global variables), are all considered as *shared data*.

Data oversharing Defining minimal shared data between compartments is challenging. Oversharing occurs when more data is exposed than necessary [24, 25], often with complex data structures where an entire object hierarchy is shared when only a few fields are needed. This increases attack surface, risking information leaks or corruption. To minimize oversharing, frameworks implement various sharing strategies: object-level [27], field-level [3, 16, 21], or byte-level [6, 9].

Interface safety A key security threat within compartmentalized applications is the attacks towards compartment interfaces, which can corrupt the data/control flows at the compartment boundaries [24, 37]. Lefeuvre et al. [25] define interface safety as the validation of all control and data flows at compartment boundaries.. When interface safety is absent, security properties enabled by compartmentalization can be compromised [19, 24, 25]. As discussed by Naranyan et al., a malicious third-party libjpeg library, even under isolation, can abuse the data/control flow to perform arbitrary writes and even execute arbitrary code in a firefox renderer that uses this library [32].

2.2 eBPF for Security Monitoring

Extended Berkeley Packet Filter (eBPF) is a powerful, in-kernel virtual machine that allows users to run sandboxed programs in the Linux kernel without changing kernel source code or loading kernel modules. eBPF programs can be attached to various kernel hooks and locations in userspace application, providing a flexible and efficient way to extend host application's functionality at runtime. Such capability makes eBPF useful for tracking program runtime states, monitoring events, and can be used for security.

There are a plethora of existing studies that demonstrated the effectiveness of eBPF in enforcing various security policies, ranging from system call filtering, network access control, to application-specific security rules [2, 13, 15]. A key advantage of eBPF is its ability to extend existing codebases non-intrusively. This is achieved through two robust tracing mechanisms: uprobes for user-space instrumentation and kprobes for kernel-space instrumentation. These probes can be strategically attached to both function entry and exit points (uprobe/uretprobe in user-space, kprobe/kretprobe in kernel-space), enabling comprehensive monitoring of function execution throughout the system.

3 Related Work

3.1 Compartment Interface Vulnerabilities

Compartment Interface Vulnerabilities (CIVs [24]) refer to vulnerabilities that exist at the interfaces between compartments and can be exploited to compromise interface safety. Such attacks are well discussed in previous compartmentalization works, yet their impact on the compartmentalization security have not been widely studied until recent research.

Butt et al. [5] developed a device driver isolation framework based on the Microdriver architecture [16]. In their approach, an isolated driver runs in a userspace process (u-driver) and interacts with a kernel counterpart (k-driver) through RPC. The authors demonstrated that a compromised u-driver could abuse calls to the k-driver or corrupt shared data structures to launch various attacks and violate interface safety.

Hu et al. [19] identified a type of memory attack called Dereference Under Influence (DUI), in which an untrusted compartment can pass data that influences memory operations in the trusted compartment. Depending on the controlled data (memory operand or address), an attacker can read sensitive data from the trusted compartment or modify its memory content through buffer overflow. Narayan et al. [32] categorized potential attacks into two broad categories: insecure data flow and insecure control flow. Both can lead to violation of interface safety. Building upon RLBox's classification, Lefeuvre et al. [24] provided a more comprehensive taxonomy of the vulnerabilities exist at the compartmentalization interface, and term these vulnerabilities as CIVs. Their work refines and extends RLBox's categories, dividing CIVs into three broad classes: data leakage, data corruption, and temporal violations. They developed an in-memory fuzzer, which mutates the data passed through the compartment interface to identify potential data-corruption related CIVs. Chien et al. developed CIVScope [7], which utilizes static analyses to identify corrupted shared data that can be used in trusted compartment's memory operations that can cause DUI style attacks for compromising both integrity and confidentiality.

3.2 Existing CIV Mitigation

Several compartmentalization approaches address CIVs, each with limitations. Butt et al. [5] developed a security monitor for Microdriver [16] to protect the kernel from compromised device drivers. Their monitor checks data invariants at each downcall to prevent data corruption CIVs. While effective for this purpose, it lacks comprehensive coverage of other CIV types (discussed in Section 5) and is tightly coupled with the Microdrivers framework, limiting its reusability. RLBox [32] takes a different approach, extending the C++ type system and providing interface wrappers to mediate data and control flow. RLBox effectively mitigates many CIVs and uses compiler errors to alert developers to potential vulnerability patterns, such as using untrusted data without validation. However, implementation can be challenging; isolating a library takes up to 3 days and requires approximately 180 lines of code changes. Additionally, RLBox requires applications to be written in C++ and needs custom validation functions for tainted data, which demands domain knowledge and can be error-prone. Our work explores eBPF-based security monitors at interface boundaries. This approach offers several advantages: unlike Butt et al.'s solution [5], eBPF provides flexible mitigation through pre-defined probe points that integrate easily with any compartmentalization framework. Compared to RLBox [32], our approach requires minimal code changes and works with any programming language. Though it may not provide the same strong guarantees as RLBox's type-based enforcement, it offers simpler deployment and configuration. Additionally, eBPF operates with kernel privilege and undergoes verification before execution; assuming a trusted kernel, the monitor cannot be bypassed by malicious userspace compartments. Finally, eBPF security policies can be dynamically updated without recompiling or restarting the application.

4 Threat Model

We assume a compartmentalized user-space application where each compartment maintains isolation of its code, stack, and heap. While the underlying compartmentalization mechanism enforces integrity, confidentiality, and availability between compartments, it does not ensure interface safety. We also consider the kernel is trusted, and hence the eBPF programs run in the kernel context cannot be tampered or compromised.

Our threat model considers an untrusted compartment U that may contain exploitable vulnerabilities. We assume an attacker can achieve arbitrary code execution within U through these vulnerabilities. The remaining compartments are considered trusted and noted as T. The attacker's goal is to compromise interface safety through U to attack T.

We consider side-channel and transient execution attacks out of scope as they represent orthogonal security challenges that require specialized solutions beyond compartmentalization [32]. In addition, we do not assume the deployment of security hardening techniques such as control flow integrity (CFI) or memory safe languages, which can provide extra security properties such as memory safety in the target application. This helps evaluating the security benefits of deploying eBPFlex.

5 CIVs Taxonomy and Mitigation Policies

In this section, we discuss CIV taxonomies and the security policies required to mitigate CIVs.

5.1 CIV Taxonomies

Existing CIV taxonomies Previous research [4, 7, 24, 32] proposed various CIV taxonomies. Narayan et al. [32] classify CIVs into two broad categories: (1) insecure data flow and (2) insecure control flow. Lefeuvre et al. [24] later refine this, categorizing CIVs into (1) data leakage and data corruption (which correspond to insecure data flow) and (2) temporal violations (which include insecure control flow and concurrency issues).

Target CIV taxonomy In this study, we adapt the CIV taxonomy defined by Narayan et al. [32] as this classification is easier to clarify what can be misused by U, e.g., shared data and the invocation of interface functions. And we classify the CIVs into (1) shared data CIVs and (2) control transfer CIVs. For shared data CIVs, we adapt the CIV categories defined by Lefeuvre et al., which include two key subcategories: (1) Data leakage: when U can read sensitive information from T through shared data to compromise T's confidentiality and (2) Data corruption: when U can corrupt shared data to compromise security properties.

Regarding control transfer CIVs, we focus on interface usage violation [36, 38, 39], in which U can abuse the invocation of interface functions exported by T to violate the usage assumption [24, 32]. We discuss it in details in Section 5.4.

5.2 Data Leakage CIVs

CIV description Data leakage CIVs occur when U extracts sensitive information from T through shared data. This happens in two main scenarios: (1) **Data oversharing**: when U receives more information than necessary. As noted in Section 2, object-level sharing commonly causes oversharing. Global and unused variables may also expose sensitive information to U; (2) **Uninitialized shared object**: objects allocated by T might contain residual sensitive data from previous memory use. This includes compiler-added padding bytes inserted between structure members to meet alignment requirements, which may contain remnants of sensitive data [24].

Data oversharing Despite isolation, compartments communicate via shared data, typically through interface function arguments or return values. Sharing occurs at either *object-level* [27, 29] or *field-level* [3, 16, 21]. In object-level sharing with deep copying, all fields reachable through an object are shared. Figure 1 shows the `ssl_ctx_st` struct used between Nginx and openssl, containing 106 fields when only one is actually used by the openssl, making the other fields unnecessarily exposed. To reduce oversharing, previous compartmentalization approaches utilized field-level [3, 9, 16, 21, 28, 33, 34] sharing or even byte-level sharing [6].

```
struct ssl_ctx_st {
    OSSL_LIB_CTX *libctx;
    const SSL_METHOD *method;
    / ... (106 total fields) */
}
```

Figure 1: Structure definition of ssl_ctx_st

Uninitialized memory and compiler-added padding Newly allocated but uninitialized memory may contain residual data from previous uses. In compartmentalized applications, a malicious compartment can access this residual data when objects are shared through interfaces. This risk increases with object-level sharing, where entire structures may be shared without proper initialization. Compiler-added padding presents another risk. To optimize memory access and meet alignment requirements, compilers insert padding bytes between or after structure members. These padding areas can contain remnants of sensitive data, causing information leaks. This has been demonstrated in cases where kernel information was exposed to userspace (CVE-2014-1444 and CVE-2016-4569) [45]. When T allocates objects with such padding and shares them with U, sensitive information may leak. This vulnerability persists even with field-level sharing, as padding bytes typically remain uninitialized.

Mitigation To mitigate data leakage CIV, all shared data allocated by T, including padding bytes, must be properly initialized before being made accessible to U. This can be achieved through several approaches: explicitly initializing all fields in the data structure, using secure memory allocation functions (e.g., calloc) that zero out memory upon allocation, and properly managing compiler-inserted padding (e.g., using memset to properly clear the allocated memory region).

Security policy To effectively mitigate data leakage CIVs, the following security policies should be enforced:

- **SP1 - U must be restricted from reading any data, except for those that are designated as readable by U.**

5.3 Data Corruption CIVs

CIV Description This type of CIV can be exploited when U can corrupt shared data that is accessed by T. Depending on how T uses the corrupted data, various attacks may occur. For example, if U corrupts an array index used by T, it could trigger an out-of-bounds memory access. Successful exploitation of this CIV requires two conditions: (1) U must have write access to the shared data, and (2) T must read and use the corrupted data without proper validation. If either condition is not met, the CIV becomes unexploitable.

Oversharing increases attack surface While not the root cause, oversharing data significantly increases the attack surface for data corruption CIVs. A common example of oversharing is object-level sharing, where entire objects are shared rather than specific fields. As shown in Figure 1, when object-level sharing is employed, an untrusted OpenSSL library may potentially corrupt any field within a shared object.

Mitigation Mitigating data corruption CIVs in general require covering the following three aspects. First, implement fine-grained sharing strategies (such as field-level sharing) instead of object-level sharing to minimize the amount of shared data. This can help reduce the attack surface. Second, correctly assign write permissions to shared data at the field level, this further reduces the attack surface to only the data that can be legitimately updated by U. Third, for any data that U has write permission to, T must perform validation before using it to ensure that expected constraints are met. Based on these mitigation strategies, we derive the following security policies for hardening the interface against data corruption CIVs.

Security policy

- **SP2 - U must be restricted from writing to any shared data except for the ones that are explicitly designated as writable by U.**
- **SP3 - T must validate all shared data updated by U before using it**

5.4 Control Transfer CIVs

CIV description Control transfer CIV can happen when U abuses the interface invocation and violate the assumption on the interface usage.

Interface usage violation The assumptions on how interface functions should be invoked can be formally expressed as a set of protocols [38, 39]. Such protocols can be defined by finite state machines (FSMs). Each FSM corresponds to one protocol. It defines a set of states and allowed transitions at each state [36]. For example, consider an *object lifecycle* [38] protocol, in which a compartment exports interfaces for managing an object's lifecycle (e.g., malloc and free). The protocol can be violated if U maliciously invokes free on an object that is still in active use. This violation may cause use-after-free in T, leading to other attacks.

Mitigation Mitigating protocol violation CIVs is challenging when U is assumed to be able to invoke any interface function at any time. One potential mitigation strategy, employed by RLBox [32],

is to allow revocation of the calling capability on the interface through an `unregister()` method. When strategically placed, e.g., after a callback is known to be no longer needed by U, this method can prevent U from abusing the interface. However, such a mechanism cannot be easily integrated into most of the existing compartmentalization frameworks. In addition to this approach, enforcing the program to follow a set of protocols (defined as FSM) may also mitigate this CIV. When an interface is invoked at an incorrect state defined by any of the FSMs, the invocation should be captured and prevented.

Security policy

- **SP4 - Interface functions must only be invoked in accordance with the protocol that defines valid interactions between T and U.**

6 Security Policy Inference

As discussed in Section 5, there are 4 policies we consider to mitigate data and control CIVs. In this section, we discuss how to infer these policies.

Inferring SP1 and SP2 SP1 and SP2 policies together ensure U cannot access unauthorized data. In eBPFlex, we control this access at the field level. To infer these policies, we determine which data fields U can legitimately access. Listing 1 shows an example from the compartmentalized ffmpeg/libwebp library. The interface function `WebPmemoryWriterInit` initializes three fields in a `WebPMemoryWriter` object: `mem`, `size`, and `max_size`. Our analysis computes write access for these fields, but since they are not read, no read access is granted. Therefore, eBPFlex applies SP1 to these fields, preventing U from reading them. The `pad` field is not accessed at all; so eBPFlex applies both SP1 and SP2 to this field.

```
1  struct WebPMemoryWriter {
2      uint8_t* mem;       // final buffer (of size 'max_size',
   ↪  larger than 'size')
3      size_t   size;      // final size
4      size_t   max_size;  // total capacity
5      uint32_t pad[1];    // padding for later use
6  };

7  void WebPMemoryWriterInit(WebPMemoryWriter* writer) {
8      writer->mem = NULL;
9      writer->size = 0;
10     writer->max_size = 0;
11 }
```

Listing 1: SP1 and SP2 inference example

Inferring SP3 SP3 prevents T from using unvalidated shared data passed from U. An SP3 policy must specify what kind of validation should be performed. We adopt the concept of *data invariants* as a way of specifying what kind of validation should be performed. Therefore inferring SP3 policies is reduced to inferring the data invariants of shared data. Common examples of data invariants include data being constants ($x = a$), data being non-zero ($x \neq 0$), and data being within a range ($a \leq x \leq b$). When such invariants hold and the operations using the checked data are atomic with the checks (i.e., no time-of-check-to-time-of-use vulnerabilities), the use of the data is considered secure. We discuss the details on inferring data invariants in Section 7.4.

Inferring SP4 SP4 ensures compartment interface usage protocols are not violated. We infer these protocols as Finite-State Machines (FSMs) using execution traces [22, 35, 36]. In eBPFlex, we apply Pradel et al.'s algorithm [35] to construct FSMs from dynamic traces. The algorithm groups function calls based on dataflow relationships, where calls are connected if one uses another's output as input. The process captures function call sequences with their parameters, return values, and memory addresses, then identifies operations forming coherent protocol sequences on the same objects. The resulting FSM represents valid state transitions, with edge weights increasing as the same patterns recur across executions. Protocol violations can then be detected by comparing new executions against this FSM. Implementation details are discussed in Section 7.5.

7 Implementation

7.1 System Workflow

Figure 2 presents the workflow of eBPFlex. Our system consists of two main phases: (1) automatic policy inference and (2) automatic policy enforcement with eBPF. The primary objective of the first phase is to infer a list of policies needed to mitigate CIVs. As discussed in Section 6, we combine static and dynamic analyses to automatically infer security policies to mitigate CIVs.

In the second phase, eBPFlex takes the generated policies as input, and automatically generates a set of eBPF programs to serve as runtime security monitors that enforce the security policies. The generated eBPF programs can be run within different eBPF runtimes, depending on the performance and security needs of the developers. In our current implementation, the generated eBPF programs can be run with two eBPF runtimes: (1). the vanilla eBPF kernel module or (2). the bpftime[44] userspace eBPF runtime.

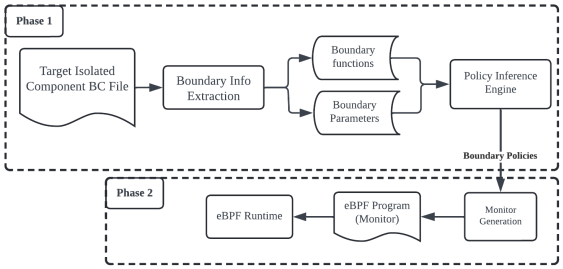

Figure 2: System Workflow

7.2 SP1/SP2 Inference and Enforcement

Policy inference As discussed in Section 6, inferring SP1/SP2 requires knowing how shared data is accessed by U and T. For that, we implement *parameters access analysis* and a set of novel optimizations for policy generation.

Parameter access analysis and policy generations Parameter access analysis is a field-sensitive analysis that computes the least access privilege on function parameters. Our implementation is based on KSplit [21], which relies on data/control dependencies in a *program dependence graph (PDG)* [12, 27] to compute parameter accesses. The analysis takes a set of interface functions and their

function parameters as input; it then utilizes the PDG to analyze how the function parameters are accessed in U. Data are assigned a `READ` label when an operation is found to read the data. Similarly, if a field is updated by an operation in U, it is assigned a `WRITE` label. Then, SP1 is enforced on the fields that do not have read access, while SP2 is enforced on the fields that do not have write access. Algorithm 1 (Appendix B) presents the overall procedure for generating access control policies from the analysis result.

Optimize policy inference Security policies SP1 and SP2 prevent unauthorized reading and writing of fields, respectively. However, in complex data structures, many fields are neither accessed by the trusted component T nor the untrusted component U, making their protection unnecessary. We next propose novel optimizations by focusing only on fields requiring protection. For SP1, we protect only fields which T writes to but U does not read from. In particular, if a field is not written by T, it does not need protection (even if U can read it) since the field cannot contain T's sensitive information. Similarly, for SP2, rather than protecting all non-writable fields, we protect only fields which T reads from but U does not write to. In particular, if a field is never read by T, there is no need for protection as T cannot be influenced by the field even if the field is corrupted by U.

Policy enforcement eBPFlex generates a collection of eBPF programs to enforce SP1 and SP2. To achieve this, it generates two eBPF functions for each target compartment interface function: one entry probe eBPF function and one return eBPF probe function. The entry probe is invoked when the instrumented interface is called, and it records the current state of the function parameters (a copy) and references to the function parameters. This information is stored in two global maps, called `param_ref_map` and `param_state_copy_map`, respectively. The `param_ref_map` is used to obtain the latest states of the function parameters, and the `param_state_copy_map` contains a snapshot of the function parameters' states when a function is called. In the return probe function, the entry parameter copies and references are retrieved from these two maps. Then SP1 and SP2 can be enforced using this information.

Enforcing SP1 To enforce SP1, eBPFlex masks non-readable states before they reach the interface and unmasks them upon return. In the entry probe, the system generates a unique identifier by combining the process ID (obtained via `bpf_get_current_pid_tgid()`), parameter index, and function ID. This identifier serves as a key to store parameter references in `param_ref_map`. Non-readable fields, identified through parameter access analysis, are obscured using bitwise XOR operations with predefined mask values. Upon function return, the return probe retrieves the parameter reference using the same unique identifier, reverses the masking process by applying identical XOR operations to restore original states, and finally removes the entry from `param_ref_map` to clean up temporary data.

```
masked_state = original_state ^ mask
original_state = masked_state ^ mask
```

Listing 2: State masking with bit operations.

Enforcing SP2 To enforce SP2, eBPFlex combines the entry and return probes to detect non-permitted updates on shared data. In the entry probe, eBPFlex computes the same unique ID as in SP1 enforcement, which serves as a unique key for storing and retrieving data in the eBPF maps. The probe stores a reference to the parameter struct in the param_ref_map, using the unique ID as the key, allowing access to the current state of the struct later in the return probe. It then creates an instance of `struct param_state_copy` and populates it with the current states of the parameter struct. The struct param_state_copy is then stored in the param_stat_copy_map, using the unique ID as the key, effectively capturing a snapshot of the non-writable fields' states at the function entry point. In the return probe, eBPFlex retrieves the reference and state copy using the unique ID for each parameter, and then accesses the current states of the non-writable fields using the retrieved parameter struct reference and compares them with the corresponding states stored in the struct param_state_copy. Any discrepancies detected during this comparison indicate unauthorized modifications to non-writable fields, and such fields are recorded. After completing the check, the probe removes the entries from both param_ref_map and param_state_copy_map.

Recovering struct type definition A key challenge in eBPF policy enforcement is accessing application struct types. The eBPF verification process and runtime constraints prevent direct importing of application headers or complex data structures, as these must be verified for safety and memory access correctness. To address this fundamental limitation, we developed an LLVM static pass that extracts struct types from interface functions and uses debugging information to recover their definitions. This type regeneration process is integrated directly into our eBPF monitor generation pipeline and works for both kernel and userspace enforcement environments.

7.3 Addressing Concurrency

In the previous design of SP1 and SP2 enforcement, object states are recorded or erased at entry points of compartment boundaries and subsequently checked or recovered at exits. This enforcement mechanism works fine when running in a single-thread setting. However, this design would produce false positives (incorrectly flagging benign data accesses) in concurrent, multi-threaded environments. As an example, imagine a shared object containing a field `counter` and there are two interface functions `fun1` and `fun2`, running concurrently by two threads T1 and T2, respectively. Function `fun1` does not modify counter while `fun2` does. In this case, T1 records the old counter value and sees that counter was updated (because of T2), and thus an alarm is triggered, as it expects no modification on this field. To mitigate this issue, we develop novel techniques for SP1-enforcement and for relaxing the policy computation for both SP1 and SP2.

Reference-counted masking (SP1) SP1 is implemented by masking a state before the control is transferred through the interface. Such masking operation can cause multiple issues during concurrency. First, if there are two threads masking the same state and using the same mask value, we must ensure that the masking is not performed twice, which would revert the state to the original value and render the enforcement moot. To address this, we employ

a reference-counted masking mechanism protected by eBPF spinlocks, maintaining consistent masking across concurrent threads (see Listing 5 in Appendix C). With this mechanism, the mask is applied for a thread only if no other threads have crossed the isolation boundary. Subsequent threads, when crossing the boundary, only increase the `ref_count` without applying the mask. Upon exit from U to T, only the thread with `ref_count` being 0 will apply the unmask operation. This prevents the masking operation from being performed twice.

Second, when some state is masked by one thread, if another thread on the trusted side T accesses the masked state, it would cause the second thread's operations to use the wrong state. To address this, our analysis determines whether shared objects are not consumed by the trusted caller until all threads executing in U are returned. If this does not hold, we relax the policy to exclude all states that are read by T. This reduces SP1 protection to only the fields that are updated by T and read by U, but not those that are read by T.

Relaxing SP2 policy for concurrent accesses Concurrent updates to shared objects may lead to policy violations in threads invoking specific interfaces. The underlying cause of this issue is that SP2 policies are inferred based solely on individual interface functions and the functions they transitively invoke, under the assumption of single-threaded execution. Consequently, although inferred SP2 policies are correct for single-interface invocations, concurrent accesses may invalidate these policies.

To mitigate this problem, we propose a relaxation of the SP2 policy inference approach. Rather than analyzing the access patterns of individual interface functions to shared objects, we analyze the collective access patterns within the entire U compartment. If a field is accessed by U in any of its code, we determine that SP2 cannot reliably hold for this field. Using this revised inference method for the false positive example, the inferred policy correctly specifies that SP2 should not be enforced on the `counter` field across both interface functions, as `fun1` does not modify the field. This adjustment effectively resolves the identified policy violation, ensuring that SP2 policy enforcement remains correct even in concurrent execution scenarios.

7.4 SP3 Inference and Enforcement

Policy inference eBPFlex leverages Daikon to generate SP3 policies by examining program execution traces to identify data invariants at function boundaries. To infer policies, we run Daikon's default templates on interface data while collecting execution traces from diverse workloads. However, not all inferred invariants require enforcement as SP3 policies.

We implement a two-stage filtering process to identify only the security-critical invariants. First, we perform *interface filtering* to select invariants relevant to compartment boundaries. This stage retains only invariants for interface function parameters and return values while eliminating those for internal variables or overfitted to specific execution paths. Second, we apply *shared field filtering* based on our static analysis results. The key insight is that SP3 enforcement is only necessary for fields that are both: (1) shared between compartments according to our access analysis, and (2) potentially capable of causing integrity violations. Specifically, we enforce SP3 only on fields that U can write to and T subsequently reads, as these represent the actual attack surface for data corruption CIVs. Private fields that are never shared across compartment boundaries do not require validation, regardless of any invariants Daikon might infer for them. This targeted approach significantly reduces the number of SP3 policies while maintaining security coverage. For example, if a field is marked as non-writable by our SP2 analysis, any invariants on that field are excluded from SP3 enforcement since U cannot corrupt it. Similarly, fields that T never reads are excluded even if U can modify them, as they cannot influence T's execution.

To ensure workload quality for accurate invariant inference, we use coverage tests (`lcov`) to verify that our workloads adequately exercise the target library code and compartment APIs. Section 8.2 details the coverage metrics and policy reduction achieved through our filtering approach.

SP3 Filtering Example: Nginx/libpcre Interface Consider the `pcre_exec` function from the Nginx/libpcre interface. This function performs pattern matching and takes multiple parameters:

```
int pcre_exec(const pcre *code, const pcre_extra
↪ *extra, const char *subject, int length, int
↪ startoffset, int options, int *ovector, int
↪ ovecsize);
```

Daikon initially infers invariants for all parameters. For instance:

```
// Initial Daikon invariants (simplified)
code != NULL
extra == NULL  // Nginx typically doesn't use extra
subject != NULL
length >= 0
0 <= startoffset <= length
options == 0   // Nginx uses default options
ovector != NULL
ovecsize >= 30
```

However, our static analysis reveals that libpcre (the untrusted compartment U) can only write to specific fields within the `pcre` structure:

```
struct pcre {
    uint32_t magic;        // Read-only by U
    size_t size;           // Read-only by U
    uint32_t options;      // Read-only by U
    uint16_t top_bracket; // Writable by U, read by T
    uint16_t name_count;  // Writable by U, not read
↪   by T
    // ... other fields
};
```

Our shared field filtering applies the following rules:

(1) **Parameters**: The scalar parameters (`length`, `startoffset`, `options`) are passed by value from Nginx to libpcre and cannot be modified by U. These invariants are excluded.
(2) **Read-only fields**: Fields like `magic`, `size`, and `options` within the `pcre` structure are protected by SP2 (non-writable by U). Invariants on these fields are excluded.
(3) **Write-only fields**: The field `name_count` can be modified by U but is never read by Nginx. This invariant is excluded as corruption cannot affect T.
(4) **Security-critical fields**: Only `top_bracket` is both writable by U and read by T, requiring SP3 validation.

After filtering, only these SP3 policies remain:

```
// Final SP3 policies for pcre_exec
code != NULL              // Prevent NULL dereference
code->top_bracket <= 65   // Prevent buffer overflow
↪ in ovector
```

This reduction from 8 initial invariants to 2 security-critical policies demonstrates how our shared field analysis eliminates unnecessary validation overhead while maintaining protection against actual integrity threats.

7.5 SP4 Inference and Enforcement

Enforcing SP4 requires precise specification of an interface-usage protocol, its states, and the permitted transitions between the states. The enforcement of such a policy is based on two critical components: (1) tracking the current state and (2) determining whether an interface function invocation is permitted in that state.

SP4 inference We adapted Pradel's algorithm [36] to infer object lifecycle protocols. This approach requires two essential components: a tracer module to collect program dynamic traces and an inference module to generate policies. The tracer module captures execution traces of interface function calls using eBPF implementation. It inserts probes that record function entries, exits, arguments, and return values during execution (Table 1).

Call Type	Function	Key Arguments/Return	Object Address
CALL	pcre_compile	pattern="pattern"	-
RETURN	pcre_compile	pcre*	0x55de6a4df0
CALL	pcre_exec	code=0x55de6a4df0	0x55de6a4df0
RETURN	pcre_exec	1 (match found)	-
CALL	pcre_free	ptr=0x55de6a4df0	0x55de6a4df0
RETURN	pcre_free	void	-

Table 1: Example trace of libpcre API calls

After collecting traces, the next step is to infer an FSM from the traces.The FSM construction begins with object identification, where we detect object creation events by identifying when a function returns a pointer. For example, when pcre_compile returns a memory address 0x55de6a4df0, this address becomes our object identifier for tracking throughout its lifecycle. We then perform trace filtering, extracting only function calls that operate on this specific memory address, thereby constructing a coherent sequence such as pcre_compile → pcre_exec → pcre_free. We create FSM states by mapping each unique function in our filtered traces to a distinct state. In the libpcre example, three states are identified: `pcre_compile`, `pcre_exec`, and `pcre_free`. We then add directed transitions between consecutive function calls based on timestamp and dataflow. For instance, if `pcre_compile` is followed by `pcre_exec` on the same object, we create a transition between these states. To capture transition frequency, we assign weights by counting each transition's occurrences in the traces, helping distinguish common patterns from rare sequences. We identify initial states as functions that allocate resources (returning pointers) and final states as those that deallocate resources (like `pcre_free`). To reduce noise, we prune transitions with weights below a threshold (set to 5 in our experiments, though this is adjustable).

The resulting FSM represents the object lifecycle protocol: objects must be initialized with pcre_compile before using pcre_exec, and must eventually be freed with pcre_free to prevent memory leaks (Figure 3). This protocol is then enforced through eBPF probes that track each object's state and verify that all transitions follow the inferred FSM, flagging violations such as using uninitialized objects or failing to free allocated memory.

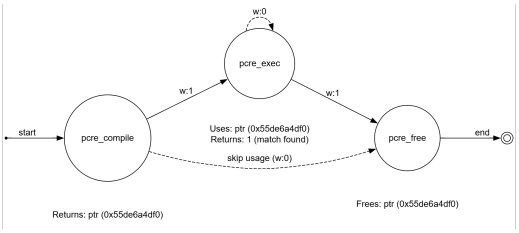

Figure 3: FSM of PCRE library interface inferred from traces.

SP4 enforcement Our implementation uses a global state map to track protocol states and verify that interface function invocations from untrusted compartments (U) conform to the correct state ordering. From a high-level, this state-tracking approach can be generalized to protect any interface function that requires specific initialization or state prerequisites. The key components are: (1) A BPF hash map that associates object instances with their states, where the key is typically the object's memory address and the value represents its current state in the FSM; (2) eBPF probes at state-changing points (like object initialization, destruction, or mode transitions) that update the state map; and (3) eBPF probes at interface function entry points that verify state correctness before allowing the function to proceed. We use the Nginx/libpcre interaction as an example to demonstrate the enforcement. Nginx uses the libpcre library for regular expression operations through three key functions: pcre_compile (which initializes a regex pattern), pcre_exec (which performs pattern matching), and pcre_free (which deallocates the regex object). A control transfer vulnerability can be exploited if pcre_exec is called on an uninitialized pattern or if a regex object is used after being freed, potentially leading to crashes or memory corruption. To prevent such illegal interface function invocations, we implement state tracking using eBPF through three key steps: (1) Creation Tracking: A BPF hash map records when pcre_compile returns a new regex object; (2) Usage Verification: When pcre_exec is invoked, the kprobe handler verifies the regex object exists in the map and is in a valid state; and (3) Deallocation Tracking: When pcre_free is called, the object's state is updated to prevent further use.

7.6 Enforcement Configurations

Enforcement modes When a policy violation is detected, eBPFlex can operate in two modes: (1). **Warning Mode**: In this mode, the execution of the program continues, but a log of the violation is recorded in the system tracing pipe. This allows for monitoring and analysis of potential security issues without interrupting the program's operation; (2). **Protection Mode**: In this more stringent mode, eBPFlex terminates program execution immediately upon detecting a violation. This is implemented by sending a `SIGKILL` signal to the process using the `bpf_send_signal` API.

Alternative eBPF Runtime The Linux eBPF module incurs some overhead due to context switches between user and kernel space when running uprobe/uretprobe. The switching overhead can be reduced by using userspace eBPF runtimes, which runs the probes entirely in userspace. eBPFlex employes bpftime [44] as an alternative eBPF runtime. Similar to kernel eBPF module, all the eBPF programs

require verification before being attached. Bpftime offers two verification methods: it can either use PREVAIL [17] or the Linux kernel verifier. In common case, bpftime can achieve up to 10x better performance for uprobes compared to kernel uprobes [44]. However, there is a performance/security tradeoff for using userspace eBPF runtimes. First, bpftime may introduce different security concerns compared to the kernel eBPF module. For example, its reliance on userspace JIT compilation and binary rewriting may increase the attack surface because flaws in the JIT or verification process could be exploited to compromise the host process. In addition, bpftime requires extra setup to ensure proper integration and secure operation within the host environment. Finally, it cannot be used to instrument kernel code (kprobe/kretprobe), which eBPFlex can potentially support. As a result, we evaluate the performance on both kernel eBPF and bpftime in Section 8.

8 Evaluation

We aim to answer three key questions in the evaluation: (Q1) the performance overhead of eBPFlex (Section 8.1), (Q2) the protection coverage provided by eBPFlex's automatic inference (Section 8.2) and (Q3) the security benefits of deploying eBPFlex (Section 8.3).

Setup Most evaluations are performed on a machine equipped with 2 Intel(R) Xeon(R) Gold 6136 CPUs running at 3.00GHz. The system architecture is x86_64, with 64 KiB of L1d cache, 64 KiB of L1i cache, 2 MiB of L2 cache, and 49.5 MiB of L3 cache. The OS installed on the machine is Ubuntu 20.04, which provides support for eBPF. For the Nginx evaluation specifically, we used an AWS EC2 medium instance (2 vCPU + 4GB memory) running Ubuntu 22.04 to better represent typical web server deployment environments.

8.1 Performance Overhead

We evaluated eBPFlex's performance using both micro-benchmarks to measure policy enforcement overhead and real-world userspace applications to assess runtime impact.

Micro-benchmarks To understand the performance characteristics of eBPFlex, we conducted micro-benchmarks on individual operations in our Nginx instrumentation. Our measurements indicate that each security policy (SP1-SP4) incurs only a few hundred nanoseconds of overhead per invocation (Table 2). SP1 enforcement involves updating a BPF map with a parameter reference and applying simple masking logic to sensitive fields, adding roughly **300ns** per invocation. SP2 performs a map update at function entry to snapshot the initial state and, at function return, a map lookup, value comparison against the snapshot, and cleanup (map deletion); this has about **580ns** overhead per invocation. SP3 simply issues a map lookup to retrieve the expected safe value and then verifies the current parameter against the data invariants, incurring only around **75ns** overhead. SP4 uses a hash map to track per-object states; each state transition requires a map lookups (approximately 70ns) to verify the object's current state and a map update (about 135ns) to record the new state and finally a check (75ns) to check the valid state transition. In sum, a typical SP4 check adds on the order of **280ns** per operation. These per-invocation costs are minimal compared to the baseline cost of invoking the probes themselves. The overhead of the instrumentation probes themselves is significantly reduced by using the bpftime userspace eBPF runtime. When using kernel eBPF module, each uprobe incurs roughly 2800ns overhead, and each uretprobe adds about 3010ns overhead. By using bpftime, the overhead on uprobe and uretprobe are to about 310ns and 379ns respectively.

Category	Operation	Time (ns)
Map Setup	BPF_HASH map creation	120
Map Operations	Map update	135
	Map read	70
Check	Value comparison	5
	Violation logging (bpf_trace_printk)	1348
Map Cleanup	Map delete	85
entry probe	uprobe	2802
return probe	uprobe	3010
bpftime entry probe	uprobe	310
bpftime return probe	uretprobe	379

Table 2: Micro-benchmark measurements for eBPFlex operations

Benchmark overhead Next, we conduct performance overhead measurement in 4 benchmarks. These benchmarks and the target isolated libraries are shown in Table 3. The main reason we selected these benchmarks/library interfaces, is because these libraries contain known CVEs that make them vulnerable and enable attackers to launch arbitrary code execution (shown in Table 6 in the appendix). For each benchmark, we measure three metrics: (1) original performance, (2) performance overhead using the kernel eBPF module and (3) performance overhead when using the bpftime eBPF runtime [44].

Benchmark	Untrusted Library
Nginx (Web server)	libpcre
Nginx (Web server)	openssl
Memcached (Memory caching)	internal hashtable
Rsync (File Transfer)	options parser
FFmpeg (Image Processing)	libavfilter

Table 3: Benchmarks and their corresponding untrusted libraries

Nginx We measured eBPFlex performance overhead when enforcing policies on two libraries: *openssl* (for secure HTTPS connections) and *libpcre* (for pattern matching in URL rules). For openssl, there are 34 interface functions across 75 call sites, while for libpcre, there are 14 interface functions across 18 call sites (PCRE2 only). Figure 4 shows our performance results comparing both eBPFlex and bpftime approaches. We evaluated the overhead with both single-thread and multi-thread (8 threads) configurations. We sent HTTPS requests for files ranging from 32KB to 2MB, with HTTPS enabled (triggering openssl) and rewrite rules configured (triggering libpcre). In single-threaded tests, smaller files showed higher overhead. With 32KB files, baseline performance was 280 requests/second. When instrumenting openssl, eBPFlex dropped to 240 requests/second (13% overhead), while bpftime achieved 274 requests/second (only 2-3% overhead). For libpcre, eBPFlex showed 7% overhead (260 requests/second) versus bpftime's 3% (272 requests/second). As file sizes increased to 2MB, overhead decreased to 4-5% for eBPFlex and 1-2% for bpftime, as disk I/O became the bottleneck. With 8 threads processing 32KB files, eBPFlex showed approximately 8% overhead for openssl (1650 vs. baseline 1800 requests/second) and 5% for libpcre (1700 requests/second). bpftime performed better with only 2% overhead for openssl (1764 requests/second) and 3% for libpcre (1746 requests/second). Larger file

sizes again showed lower overhead for both approaches. These results show that bpftime offers significant performance advantages by eliminating context switching. bpftime's 2-3% overhead makes it particularly suitable for performance-critical production environments.

Rsync overhead We evaluated eBPFlex's performance overhead on Rsync by measuring the time taken to synchronize different numbers of files (1, 10, and 100 files). All measurements represent averages across 10 independent runs. As shown in Figure 5, the overhead is consistently low across all workloads. For a single file transfer, the baseline completion time averages 0.4 seconds, with eBPFlex adding only 2% overhead (0.408 seconds). This small overhead pattern continues as we increase the workload. For 10 files, the average time increases from 0.7 to 0.714 seconds (2% overhead), and for 100 files, from 3.0 to 3.09 seconds (3% overhead). Overall, eBPFlex achieves small overheads below 3% across all file transfer scenarios.

Memcached overhead For Memcached, we measured the performance overhead using the memcslap benchmarking tool, which provided both Get (read) and Set (write) workloads. The overhead was measured by the time required to accomplish a workload. We tested with different numbers of key-value pairs (1000, 5000, and 10000) to understand how the overhead scaled with the workload size. The results are shown in Figure 6 and Figure 7.

For the GET workload with eBPFlex, the overhead increased with workload size: 8% at 1000 pairs (5.0 to 5.4 seconds) and 14% at 10000 pairs (30.0 to 34.2 seconds). bpftime significantly reduced this overhead to just 1.6% at 1000 pairs and 2.8% at 10000 pairs. The SET workload showed higher overhead due to more frequent interface function invocations. With eBPFlex, we observed 39% overhead at 1000 pairs (12.0 to 16.7 seconds), growing to 83% at 10000 pairs (65.0 to 119.0 seconds). bpftime dramatically reduced this to just 7.8% at 1000 pairs and 16.6% at 10000 pairs. Both implementations perform identical SP1/SP2 and SP3 policy checks, but bpftime's substantial performance advantage comes from eliminating context switches between user and kernel space during policy enforcement. This improvement is particularly noticeable for SET operations, which invoke interface functions more frequently.

FFmpeg overhead To evaluate performance overhead, we created three video processing workloads of increasing length (and number of frames): (1) a 10-second video: a short 1080p test clip (300 frames, H.264 format); (2) a 1-minute video: the same 1080p content extended to 60 seconds (1,800 frames) and (3) a 1-hour video: a long-duration 1080p sequence (108,000 frames), created by looping the test clip. Each workload was processed with FFmpeg's command-line tool, applying the crop and scale filters in sequence. The overhead is shown in Figure 8, each measured with 10 runs on the workloads. Overall, we observe that the instrumentation overhead introduced by eBPFlex scales consistently with the workload size, remaining moderate relative to total processing time. Specifically, eBPFlex adds approximately 3.9 seconds (324%) overhead to the baseline of 1.202 seconds for the shortest workload. For the 1-minute workload, with a baseline processing time of 10.211 seconds, the overhead reduces to roughly 4.9 seconds (48%). For the longest workload (1 hour), which has a baseline time of approximately 593.895 seconds, the overhead further decreases to about 35 seconds (5.9%). We note that shorter videos show a large overhead as the instrumentation time is comparable to the total video-processing time; for longer-duration workloads, the instrumentation overhead becomes relatively small compared to the total processing time.

Performance overhead summary Our evaluation indicates that eBPFlex's performance overhead varies across different application scenarios and workloads. For network applications like Nginx, kernel runtime shows moderate overhead (5-13%) when enforcing policies on openssl and libpcre libraries, while using bpftime reduces the overhead to 2-3%. Rsync instrumentation has consistently low overhead below 3% across all file transfer scenarios. For memcached, GET operations show 8-14% overhead with kernel runtime versus 1.6-2.8% with bpftime, while SET operations show 39-83% kernel overhead compared to 7.8-16.6% with bpftime. FFmpeg evaluation shows that overhead can vary with workload duration, showing higher relative impact for shorter tasks (324% for 10-second videos) and lower impact for longer processing tasks (5.9% for 1-hour videos). These results suggest that bpftime may offer performance advantages by eliminating user-kernel space context switching, particularly for applications with frequent cross-compartment interactions, though the benefit appears to diminish when other factors like network latency or disk I/O become the dominating part of the overall workload time.

8.2 Policy Coverage

To evaluate how effectively eBPFlex can protect compartment interfaces through the enforced policies, we analyze the coverage for each policy across our benchmark applications. The protection coverage analysis for each policy is summarized in Table 4. For SP1 and SP2, fields fall into three categories: (1) safe fields, which do not require protection as explained in Section 7.2, (2) fields that require protection and are identified by our policy inference, and (3) fields that might need protection but are not covered due to the relaxation techniques we mentioned in Section 7.3. Our analysis shows that for SP1, most fields are safe and only 2.2% to 27.3% of fields across applications require protection against unauthorized reads. For safe fields, as discussed in our policy optimization, no protection is needed since trusted component T never writes to those fields; even if untrusted component U could read such a field, there is no sensitive information to protect. These inherently safe fields account for 75.3% of fields in PCRE, 69.8% in SSL, 54.5% in the hash table, 57.1% in the option parser, and 83.3% in libavfilter. Similarly, for SP2, between 10.4% and 63.6% of fields require protection against unauthorized modifications, while a substantial portion (67.5% for PCRE, 67.0% for SSL, 9.1% for the hash table, 28.6% for the option parser, and 54.4% for libavfilter) do not require protection because T never reads these fields; even if U modifies them unexpectedly, it would not impact T's operations.

As we can see, inherently safe fields account for the majority, which showcases the importance of novel optimizations for SP1/SP2 policy generation. Together, the protected fields and inherently safe fields account for substantial coverage: 81.8% (PCRE), 77.1% (SSL), 81.8% (hash table), 78.5% (option parser), and 85.5% (libavfilter) for SP1 protection; and 77.9% (PCRE), 77.6% (SSL), 72.7% (hash table), 71.5% (option parser), and 76.6% (libavfilter) for SP2 protection. The

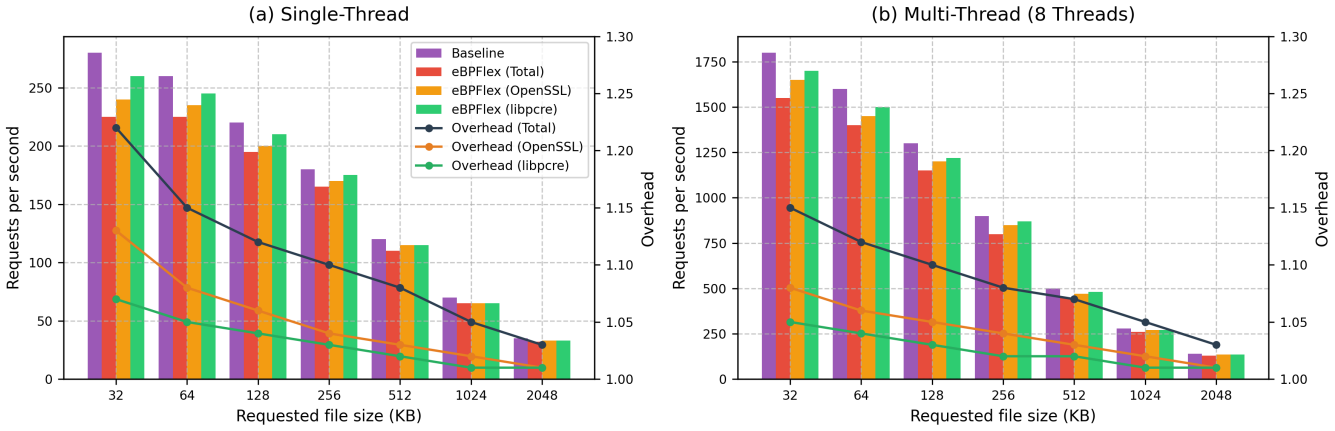

Figure 4: Nginx performance overhead with eBPFlex when isolating OpenSSL and libpcre libraries

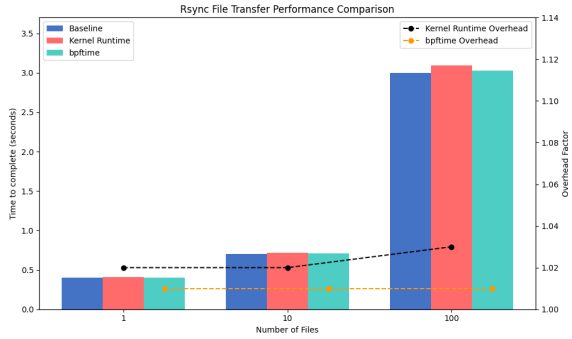

Figure 5: Rsync performance overhead with eBPFlex when isolating the option parser module

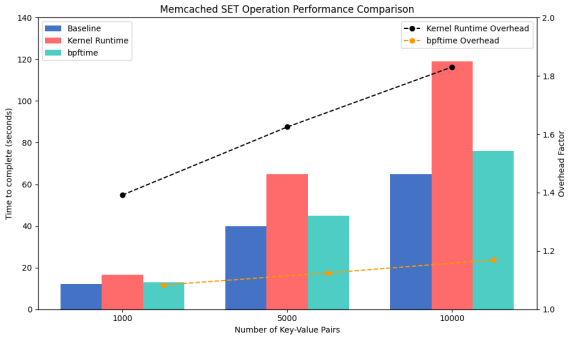

Figure 7: Memcached performance overhead with eBPFlex when isolating the internal hashtable module on the SET workload

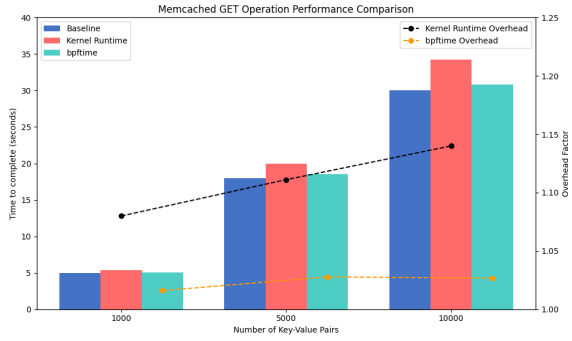

Figure 6: Memcached performance overhead with eBPFlex when isolating the internal hashtable module on the GET workload

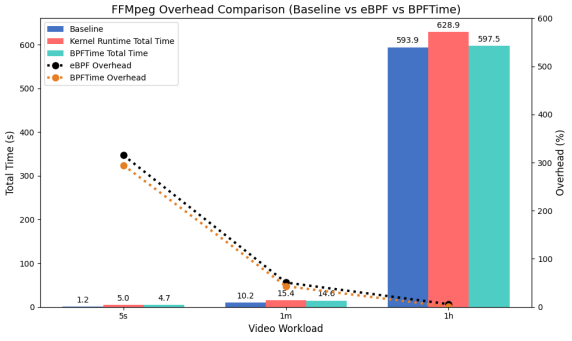

Figure 8: FFmpeg performance overhead with eBPFlex when isolating the libavfilter library

remaining fields fall into the third category, where protection is relaxed to avoid false positives in concurrent executions.

Regarding SP3 (data validation), coverage ranges from 11.7% to 57.1% of fields across applications. As discussed, Daikon can only generate invariants for fields that appear in execution traces with sufficient variation to infer meaningful constraints. To ensure the generated data invariants quality, the workloads for generating the traces must be representitive. To measure how good our workloads are, we perform code coverage analysis to measure the coverage of the library code. Specifically, we measure two key metrics: (1). API coverage and (2). library code line coverage. The former metric shows how many library interfaces are covered by the workloads and the line coverage shows the overall coverage of the library code. Overall, our test workloads achieve high API coverage (85–100%) but moderate line coverage (13–27%), consistent with previous

Metrics	PCRE	SSL	hash table	opt parser	libavfilter
No. Interface funs	13	29	10	10	41
No. Call sites	28	65	14	18	100
No. total fields	77	179	11	14	90
SP1 Policy (Protection against unauthorized reads)					
Safe fields	58 (75.3%)	125 (69.8%)	6 (54.5%)	8 (57.1%)	75 (83.3%)
Protected fields	5 (6.5%)	13 (7.3%)	3 (27.3%)	3 (21.4%)	2 (2.2%)
Total coverage	63 (81.8%)	138 (77.1%)	9 (81.8%)	11 (78.5%)	77 (85.5%)
SP2 Policy (Protection against unauthorized modifications)					
Safe	52 (67.5%)	120 (67.0%)	1 (9.1%)	4 (28.6%)	49 (54.4%)
Protected fields	8 (10.4%)	19 (10.6%)	7 (63.6%)	6 (42.9%)	20 (22.2%)
Total coverage	60 (77.9%)	139 (77.6%)	8 (72.7%)	10 (71.5%)	69 (76.6%)
SP3 Policy (Data invariant)					
SP3 fields	9 (11.7%)	21 (11.7%)	4 (36.4%)	8 (57.1%)	17 (18.9%)
No. SP3 policies	13	29	6	11	24

Table 4: Analysis of security policy coverage across benchmarks. The table shows fields that receive protection through inference, fields that are inherently safe, and their combined coverage. SP1 protects against unauthorized reads, SP2 protects against unauthorized modifications, and SP3 performs validation on interface parameters and return value. Numbers in parentheses show percentage of total fields in each category

fuzzing studies [1, 23]. Detailed coverage data can be found in Table 7 (Appendix F). Overall, for SP3 coverage, more complex or larger interfaces like libavfilter have lower coverage rate (18.9%) compared to simpler interfaces like the option parser (57.1%). Finally, we identified 83 SP3 policies across all the benchmarks. For the inferred invariants, pointer non-nullness invariants are the most frequently observed pattern. Constant-valued invariants (especially fixed return codes like 0 for success, or consistently NULL/zero parameters) are also prevalent, particularly in the libavfilter interfaces. Range constraints occur in specific cases (such as lengths or indices) but are less common overall. We show partial data invariants we inferred in Table 5 (Appendix D).

For SP4 converage, eBPFlex successfully constructed FSMs for some lifecycle protocol for 3 compartment interfaces (out of 5). In the nginx/libpcre interface, eBPFlex learned a three-state protocol corresponding to regex compilation, usage, and cleanup. The inferred FSM requires that a pattern object be initialized via pcre_compile before being used by pcre_exec, and ultimately deallocated with pcre_free. For nginx/OpenSSL interface, eBPFlex captured an SSL connection's lifecycle. An SSL object must be created via the SSL_new interface before any handshake or data transfer functions, and must be freed with the appropriate clean up interface SSL_free. This yields an FSM that enforces correct order (preventing, for instance, use of an SSL object before initialization or after free). In FFmpeg/libavfilter interface, we also observed a successful FSM inference. The dynamic traces revealed a filter graph management protocol: a filter graph is allocated and configured by the interface function avfilter_graph_alloc. After allocation, filter components are typically created and configured using avfilter_graph_create_filter. Individual filter contexts are initialized, then linked together via avfilter_link(), and the complete graph is finalized by calling avfilter_graph_config. Once filtering is completed, the filter graph and all its associated contexts are freed using avfilter_graph_free. The remaining two benchmarks did not yield meaningful FSMs from our inference.

8.3 Security Assessment

To evaluate the security benefits of eBPFlex, we construct a proof of concept (PoC) containing multiple CIV patterns and demonstrate how eBPFlex prevents their exploitation. Our PoC (Listing 3) is a compartmentalized image processing program with a malicious libjpeg library based on scenarios described in RLBox [32].

Our PoC contains three distinct CIV patterns that a malicious compartment could exploit. The first vulnerability (CIV1, lines 18-20) involves data corruption. The `my_skip_input_data` callback function receives the `num_bytes` parameter from the untrusted libjpeg without validation. A malicious library could supply a crafted value to cause integer overflow/underflow, bypassing the bounds check and triggering an out-of-bounds memory access when performing pointer arithmetic on `next_input_byte`. The second vulnerability (CIV2, lines 31-34) concerns data leakage. When passing the `cinfo` structure to library functions such as `jpeg_set_defaults`, private fields can be leaked because the entire structure is exposed to the untrusted compartment.

The third vulnerability (CIV3, lines 36-37) relates to pointer corruption. The `buffer` pointer is stored in a structure accessible to the untrusted library. A malicious library could modify this pointer value, causing the subsequent `free()` operation to target an arbitrary memory location, potentially leading to use-after-free or double-free vulnerabilities. We deployed eBPFlex with specific policies to mitigate each vulnerability. For CIV1, we enforce **SP3** to validate the `num_bytes` parameter, ensuring it falls within an expected range derived from dynamic invariant analysis. To prevent CIV2, we apply **SP1** to restrict the libjpeg library from reading sensitive fields in the `cinfo` structure, particularly the private fields such as `key` and `config`. Finally, **SP2** mitigates CIV3 by preventing the untrusted library from modifying the `buffer` pointer, thus protecting the integrity of memory management operations. During testing, we verified that eBPFlex successfully detected all attempted exploits of these vulnerabilities. In warning mode, eBPFlex logged each violation with specific information about the violated policy. In protection mode, it terminated execution upon detecting the first violation, effectively preventing potential attacks. These results demonstrate eBPFlex's effectiveness in mitigating different types of CIVs through automatically inferred security policies, showing how compartmentalized applications can be hardened against interface vulnerabilities with minimal developer effort.

9 Future Work

While eBPFlex strengthens compartment interfaces via automatic policy inference and enforcement, there are several limitations in the current design that suggest directions for future work. We outline two key areas for improvement.

Improving Data Invariant Inference (SP3). Our current approach relies on Daikon's dynamic invariant detection, which is limited to predefined templates and depends heavily on workload quality. Important invariants may remain undiscovered if they do not appear in observed traces [10]. Future work could incorporate static analysis or formal methods for more comprehensive invariant detection. For example, tools such as the Houdini annotation assistant in ESC/Java can guess and verify likely invariants without extensive runtime data [14]. Other promising approaches

```c
typedef struct {
    struct jpeg_source_mgr pub; // public fields
    FILE* infile;  // source stream
    JOCTET* buffer;  // start of buffer
    JOCTET* next_input_byte;  // next byte to read
    size_t bytes_in_buffer;  // remaining bytes
    unsigned key;  // private value
    char* config;  // private config
} source_mgr;

typedef source_mgr* src_ptr;
// A callback function registered with libjpeg
void my_skip_input_data(j_decompress_ptr cinfo, long
↪ num_bytes) {
    my_src_ptr src = (my_src_ptr)cinfo->src;
// CIV1: No validation on num_bytes which comes from
// potentially malicious compartment
    if (num_bytes > 0) {
        src->pub.next_input_byte += num_bytes;
        src->pub.bytes_in_buffer -= num_bytes;
    }
}

int load_jpeg_from_memory(Image *image, unsigned char buffer,
unsigned long size) {
    struct jpeg_decompress_struct cinfo;
    /* Set parameters */
    cinfo.image_width = image->width;
    cinfo.image_height = image->height;
    cinfo.input_components = image->channels;
    cinfo.in_color_space = (image->channels == 3) ?
    JCS_RGB : JCS_GRAYSCALE;

//CIV2: The whole cinfo struct is passed to jpeg interface,
↪ and all the fields can be read
    jpeg_set_defaults(&cinfo);
    jpeg_read_header(&cinfo, TRUE);
}

int main(int argc, char *argv[]) {
    struct jpeg_decompress_struct cinfo;
    source_mgr* src;
    // Allocate memory for the input buffer
    char *buffer = malloc(FILE_BUFFER_SIZE);
    ...
    // Set up the source manager
    src = (source_mgr*)malloc(sizeof(source_mgr));
    src->buffer = buffer; // Store pointer to allocated memory
    cinfo.src = (struct jpeg_source_mgr*)src;
/*
 CIV3: The buffer pointer is stored in a field and passed to
↪ jpeg_read_header. U can modify src->buffer to point to an
↪ arbitrary location.
*/
    jpeg_read_header(&cinfo, TRUE); // Library crossing point
    // Later may free a pointer that was maliciously modified
    free(src->buffer); // Potential use-after-free or
↪ double-free
    free(src);
    jpeg_destroy_decompress(&cinfo);
}
```

Listing 3: Untrusted libjpeg exploiting CIVs

include counterexample-guided techniques for synthesizing invariants [30] and machine learning methods like Horn-ICE [11] that combine symbolic reasoning with data-driven generalization. These approaches would reduce dependence on test executions and could identify stricter data validity policies that current methods miss.

Generalizing Protocol Enforcement (SP4). Currently, SP4 enforcement in eBPFlex targets *lifecycle protocols*. However many interfaces have additional protocols, such as permission management requiring certain setup functions before operations or resource ownership protocols [38]. Failing to infer/enforce these protocols may still allow attacker to abuse control transfer CIVs. In future work, we plan to extend SP4 to infer and enforce more types of protocols.

10 Conclusion

In this paper, we presented eBPFlex, an automated tool for hardening compartment interface safety through security policy inference and enforcement. Our work addresses the challenge of CIVs by combining static and dynamic analyses to automatically infer security policies and leveraging eBPF to enforce these policies at runtime. Through comprehensive evaluation on real-world applications, we demonstrated that eBPFlex can effectively prevent various types of CIVs, while achieving acceptable overhead in certain types of applications.

References

[1] Nginx - fuzzing | project profile. https://introspector.oss-fuzz.com/project-profile?project=nginx. Accessed: 2025-03-27.

[2] Matteo Bertrone, Sebastiano Miano, Fulvio Risso, and Massimo Tumolo. Accelerating linux security with ebpf iptables. In *Proceedings of the ACM SIGCOMM 2018 Conference on Posters and Demos*, pages 108–110, 2018.

[3] Silas Boyd-Wickizer and Nickolai Zeldovich. Tolerating malicious device drivers in Linux. In *2010 USENIX Annual Technical Conference (USENIX ATC '10)*, 2010.

[4] Anton Burtsev, Vikram Narayanan, Yongzhe Huang, Kaiming Huang, Gang Tan, and Trent Jaeger. Evolving operating system kernels towards secure kernel-driver interfaces. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems*, page 166–173, Providence RI USA, Jun 2023. ACM.

[5] Shakeel Butt, Vinod Ganapathy, Michael M. Swift, and Chih-Cheng Chang. Protecting commodity operating system kernels from vulnerable device drivers. In *2009 Annual Computer Security Applications Conference*, page 301–310, Honolulu, Hawaii, USA, December 2009. IEEE.

[6] Miguel Castro, Manuel Costa, Jean-Philippe Martin, Marcus Peinado, Periklis Akritidis, Austin Donnelly, Paul Barham, and Richard Black. Fast byte-granularity software fault isolation. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP '09)*, pages 45–58, 2009.

[7] Yi Chien, Vlad-Andrei Bădoiu, Yudi Yang, Yuqian Huo, Kelly Kaoudis, Hugo Lefeuvre, Pierre Olivier, and Nathan Dautenhahn. Civscope: Analyzing potential memory corruption bugs in compartment interfaces. In *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification*, pages 33–40, 2023.

[8] Zakir Durumeric, Frank Li, James Kasten, Johanna Amann, Jethro Beekman, Mathias Payer, Nicolas Weaver, David Adrian, Vern Paxson, Michael Bailey, and J. Alex Halderman. The matter of heartbleed. In *Proceedings of the 2014 Conference on Internet Measurement Conference*, page 475–488, Vancouver BC Canada, November 2014. ACM.

[9] Úlfar Erlingsson, Martín Abadi, Michael Vrable, Mihai Budiu, and George C. Necula. XFI: Software Guards for System Address Spaces. In *Proceedings of the 7th Symposium on Operating Systems Design and Implementation (OSDI '06)*, pages 75–88, 2006.

[10] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, December 2007.

[11] P. Ezudheen, Daniel Neider, Deepak D'Souza, Pranav Garg, and P. Madhusudan. Horn-ice learning for synthesizing invariants and contracts. *Proc. ACM Program. Lang.*, 2(OOPSLA), October 2018.

[12] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. The Program Dependence Graph and its Use in Optimization. *ACM Transactions on Programming Languages and Systems*, 9(3):319–349, 1987.

[13] William Findlay, Anil Somayaji, and David Barrera. bpfbox: Simple precise process confinement with ebpf. page 13, 2020.

[14] Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *FME 2001: Formal Methods for Increasing Software Productivity*, pages 500–517, 2001.

[15] Guillaume Fournier, Sylvain Afchain, and Sylvain Baubeau. Runtime security monitoring with ebpf. In *17th SSTIC Symposium sur la Sécurité des Technologies de l'Information et de la Communication*, 2021.

[16] Vinod Ganapathy, Matthew J Renzelmann, Arini Balakrishnan, Michael M Swift, and Somesh Jha. The design and implementation of microdrivers. In *ACM SIGARCH Computer Architecture News*, volume 36, pages 168–178, 2008.

[17] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. Simple and precise static analysis of untrusted linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 1069–1084, 2019.

[18] Makenzie Holland. Explaining the largest IT outage in history and what's next, 2024.

[19] Hong Hu, Zheng Leong Chua, Zhenkai Liang, and Prateek Saxena. *Identifying Arbitrary Memory Access Vulnerabilities in Privilege-Separated Software*, volume 9327 of *Lecture Notes in Computer Science*, page 312–331. Springer International Publishing, Cham, 2015.

[20] Yongzhe Huang, Kaiming Huang, Matthew Ennis, Vikram Narayanan, Anton Burtsev, Trent Jaeger, and Gang Tan. Sok: Understanding the attack surface in device driver isolation frameworks. (arXiv:2412.16754), December 2024. arXiv:2412.16754.

[21] Yongzhe Huang, Vikram Narayanan, David Detweiler, Kaiming Huang, Gang Tan, Trent Jaeger, and Anton Burtsev. KSplit: Automating device driver isolation. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 613–631, Carlsbad, CA, July 2022. USENIX Association.

[22] Ivo Krka, Yuriy Brun, and Nenad Medvidovic. Automatic mining of specifications from invocation traces and method invariants. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 178–189, 2014.

[23] GitHub Security Lab. Fuzzing sockets: Apache http, part 1: Mutations. https://github.blog/security/vulnerability-research/fuzzing-sockets-apache-http-part-1-mutations/. Accessed: 2025-03-27.

[24] Hugo Lefeuvre, Vlad-Andrei Bădoiu, Yi Chien, Felipe Huici, Nathan Dautenhahn, and Pierre Olivier. Assessing the Impact of Interface Vulnerabilities in Compartmentalized Software, January 2023. arXiv:2212.12904 [cs].

[25] Hugo Lefeuvre, Nathan Dautenhahn, David Chisnall, and Pierre Olivier. Sok: Software compartmentalization. *arXiv preprint arXiv:2410.08434*, 2024.

[26] Soo Yee Lim, Sidhartha Agrawal, Xueyuan Han, David Eyers, Dan O'Keeffe, and Thomas Pasquier. Securing monolithic kernels using compartmentalization. *arXiv preprint arXiv:2404.08716*, 2024.

[27] Shen Liu, Gang Tan, and Trent Jaeger. PtrSplit: Supporting General Pointers in Automatic Program Partitioning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*, pages 2359–2371, 2017.

[28] Yandong Mao, Haogang Chen, Dong Zhou, Xi Wang, Nickolai Zeldovich, and M Frans Kaashoek. Software fault isolation with API integrity and multi-principal modules. page 14.

[29] Derrick McKee, Yianni Giannaris, Carolina Ortega Perez, Howard Shrobe, Mathias Payer, Hamed Okhravi, and Nathan Burow. Preventing kernel hacks with hakc. In *Proceedings 2022 Network and Distributed System Security Symposium. NDSS*, volume 22, pages 1–17, 2022.

[30] Anders Miltner, Saswat Padhi, Todd Millstein, and David Walker. Data-driven inference of representation invariants. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2020, page 1–15, New York, NY, USA, 2020. Association for Computing Machinery.

[31] Divya Muthukumaran, Trent Jaeger, and Vinod Ganapathy. Leveraging" choice" to automate authorization hook placement. In *Proceedings of the 2012 ACM conference on Computer and communications security*, pages 145–156, 2012.

[32] Shravan Narayan, Craig Disselkoen, Tal Garfinkel, Nathan Froyd, Eric Rahm, Sorin Lerner, Hovav Shacham, and Deian Stefan. Retrofitting fine grain isolation in the firefox renderer. In *Proceedings of the 29th USENIX Conference on Security Symposium*, pages 699–716, 2020.

[33] Vikram Narayanan, Abhiram Balasubramanian, Charlie Jacobsen, Sarah Spall, Scott Bauer, Michael Quigley, Aftab Hussain, Abdullah Younis, Junjie Shen, Moinak Bhattacharyya, and Anton Burtsev. LXDs : Towards Isolation of Kernel Subsystems. In *2019 USENIX Annual Technical Conference (USENIX ATC '19)*, 2019.

[34] Vikram Narayanan, Yongzhe Huang, Gang Tan, Trent Jaeger, and Anton Burtsev. Lightweight Kernel Isolation with Virtualization and VM Functions. In *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE '20)*, pages 157–171, 2020.

[35] Michael Pradel. Dynamically inferring, refining, and checking api usage protocols. In *Proceedings of the 24th ACM SIGPLAN conference companion on Object oriented programming systems languages and applications*, pages 773–774, 2009.

[36] Michael Pradel and Thomas R Gross. Automatic generation of object usage specifications from large method traces. In *2009 IEEE/ACM International Conference on Automated Software Engineering*, pages 371–382. IEEE, 2009.

[37] Niels Provos, Markus Friedl, and Peter Honeyman. Preventing privilege escalation. In *12th USENIX Security Symposium (USENIX Security 03)*, 2003.

[38] Leonid Ryzhyk, Peter Chubb, Ihor Kuz, and Gernot Heiser. Dingo: Taming Device Drivers. In *Proceedings of the 4th ACM European Conference on Computer Systems (EuroSys '09)*, pages 275–288, 2009.

[39] Leonid Ryzhyk, Ihor Kuz, and Gernot Heiser. Formalising device driver interfaces. In *Proceedings of the 4th workshop on Programming languages and operating systems*, page 1–5, Stevenson Washington, October 2007. ACM.

[40] J.H. Saltzer and M.D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, September 1975.

[41] Rui Shu, Peipei Wang, Sigmund A Gorski III, Benjamin Andow, Adwait Nadkarni, Luke Deshotels, Jason Gionta, William Enck, and Xiaohui Gu. A study of security isolation techniques. *ACM Computing Surveys (CSUR)*, 49(3):1–37, 2016.

[42] Michael M Swift, Steven Martin, Henry M Levy, and Susan J Eggers. Nooks: An Architecture for Reliable Device Drivers. In *Proceedings of the 10th workshop on ACM SIGOPS European workshop*, pages 102–107, 2002.

[43] Luca Wilke, Jan Wichelmann, Mathias Morbitzer, and Thomas Eisenbarth. Sevurity: No security without integrity: Breaking integrity-free memory encryption with minimal assumptions. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1483–1496. IEEE, 2020.

[44] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, XiaoZheng Lai, and Andrew Quinn. bpftime: userspace ebpf runtime for uprobe, syscall and kernel-user interactions. *arXiv preprint arXiv:2311.07923*, 2023.

[45] Sebastian Österlund, Koen Koning, Pierre Olivier, Antonio Barbalace, Herbert Bos, and Cristiano Giuffrida. kmvx: Detecting kernel information leaks with multi-variant execution. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, page 559–572, Providence RI USA, April 2019. ACM.

A eBPF SP4 enforcement example

```
// BPF map to track JPEG Source Manager initialization states
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 10000);  // Support multiple concurrent operations
    __type(key, u64);           // Address of SourceManager as key
    __type(value, u32);         // Initialization state
} jpeg_sm_state SEC(".maps");

// Track initialization at JPEGSourceManager return
SEC("kretprobe/JPEGSourceManager")
int track_init(struct pt_regs *ctx) {
    void *source = (void *)PT_REGS_RC(ctx);
    if (!source)
        return 0;

    // Use source manager's address as key
    u64 key = (u64)source;
    u32 state = 1;  // Mark as initialized
    bpf_map_update_elem(&jpeg_sm_state, &key, &state, BPF_ANY);
    return 0;
}

// Verify state before FillInputBuffer execution
SEC("kprobe/FillInputBuffer")
int check_callback(struct pt_regs *ctx) {
    void *compress_info = (void *)PT_REGS_PARM1(ctx);
    if (!compress_info)
        return -EINVAL;

    // Get source manager from compress_info
    void *source = (void *)compress_info->src;
    u64 key = (u64)source;

    u32 *state = bpf_map_lookup_elem(&jpeg_sm_state, &key);
    if (!state || *state != 1) {
        bpf_trace_printk("Security violation: Uninitialized SourceManager %llx\n", key);
        return -EPERM;
    }
    return 0;
}
```

Listing 4: eBPF implementation with per-instance state tracking

Input: Boundary Info, Program Dependence Graph (PDG)
Output: Set of inferred access control policies
Procedure *ParameterAccessAnalysis(PDG, BoundaryInfo)*:
  Compute access patterns on boundary API parameters by U;
  return *Access patterns*;
policies ← ∅;
accessPatterns ← ParameterAccessAnalysis(PDG, BoundaryInfo);
foreach *pattern* ∈ *accessPatterns* **do**
  policy ← TranslateToPolicy(pattern);
  policies ← policies ∪ policy;
end
return *policies*;

Algorithm 1: Policy inference steps

B SP1/SP2 Inference Step

C SP1 Lock Mechanism to Ensure Masking Correctness

```
struct mask_state {
    struct bpf_spin_lock lock;
    u32 ref_count;
    u64 original_value;
};

void entry_probe(struct object *obj) {
    struct mask_state *state = bpf_map_lookup(&mask_map, obj);
    if (!state) return;

    bpf_spin_lock(&state->lock);
    if (state->ref_count == 0) {
        state->original_value = obj->protected_field;
        obj->protected_field ^= MASK;
    }
    state->ref_count++;
    bpf_spin_unlock(&state->lock);
}

void exit_probe(struct object *obj) {
    struct mask_state *state = bpf_map_lookup(&mask_map, obj);
    if (!state) return;

    bpf_spin_lock(&state->lock);
    state->ref_count--;
    if (state->ref_count == 0) {
        obj->protected_field ^= MASK;
    }
    bpf_spin_unlock(&state->lock);
}
```

Listing 5: SP1 enforcement considering concurrency

D SP3 Invariant Examples

Interface Function	Invariants (Grouped)
libpcre	
pcre_compile	pattern ≠ null tableptr == null return ≠ null
pcre_exec	code ≠ null 0 ≤ startoffset ≤ length extra == null return ≥ -1
pcre_free	ptr ≠ null
libavfilter	
avfilter_graph_alloc	return ≠ null
avfilter_graph_create_filter	filter ≠ null opaque == null return == 0
avfilter_link	src_ctx ≠ null and dst_ctx ≠ null srcpad == 0 and dstpad == 0 return == 0
avfilter_graph_config	graph ≠ null return == 0
av_buffersrc_add_frame	src_ctx ≠ null frame ≠ null return == 0
av_buffersink_get_frame	sink_ctx ≠ null frame ≠ null return ≤ 0
avfilter_graph_free	graph ≠ null

Table 5: SP3 invariants inferred for libpcre and libavfilter interfaces

E Library CVE List

Past CVEs for the target library allowing arbitrary code execution.

Library/Component	CVEs (Arbitrary Code Execution)	Affected Versions
libpcre	CVE-2007-1659, CVE-2007-1660, CVE-2007-1661, CVE-2007-1662, CVE-2007-4766, CVE-2007-4767, CVE-2007-4768, CVE-2008-0674, CVE-2008-2371, CVE-2014-8964, CVE-2014-9769, CVE-2015-3210, CVE-2015-3217, CVE-2015-5073, CVE-2015-8385, CVE-2015-8386, CVE-2015-8388, CVE-2015-8391, CVE-2016-3191, CVE-2017-7245, CVE-2017-7246, CVE-2020-14155	Versions prior to 8.44 (various vulnerabilities fixed up to 8.44)
OpenSSL	CVE-2002-0656, CVE-2003-0544, CVE-2007-5135, CVE-2014-0195, CVE-2014-8176, CVE-2016-2108, CVE-2016-6309, CVE-2022-3602	Versions before 0.9.6d, before 1.0.1o/1.0.2c, 1.1.0a, and 3.0.0–3.0.6
FFmpeg libavfilter	CVE-2023-49501, CVE-2023-51794, CVE-2023-51796, CVE-2023-51797, CVE-2023-51798	Versions up to 6.0/6.1 (patched mid-2023)
Memcached (internal hashtable)	CVE-2016-8704, CVE-2016-8705, CVE-2016-8706	Versions before 1.4.32
Rsync (options parser)	CVE-2003-0962, CVE-2024-12084	Versions before 2.5.7 and 3.2.7 (fixed in 3.2.8)

Table 6: Arbitrary Code Execution CVEs for Isolated Libraries

F SP3 Workload Coverage

Benchmark	API Coverage (%)	Line Coverage (%)
Nginx/PCRE+libssl	95	21.0
Memcached/hash table	100	27.0
Rsync/opt parser	100	21.7
FFmpeg/avfilter	93	13.0

Table 7: Workload coverage metrics for benchmark applications